

COMS W4156 Advanced Software Engineering (ASE)

November 14, 2023

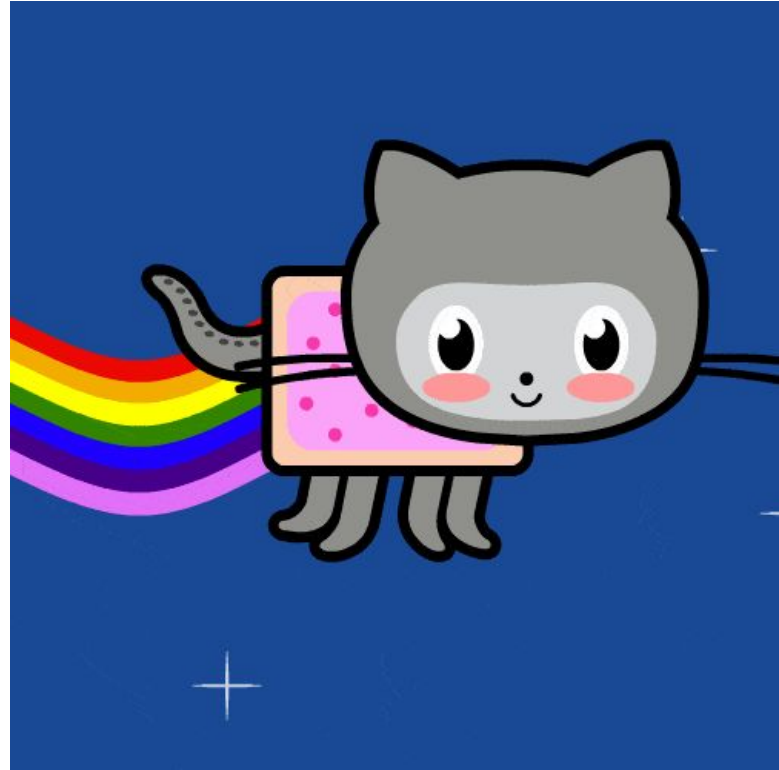


Feedback on First Assessment

Not done grading yet

#1 advertising: Some issues with distinguishing advertisers and/or opt-out clients from other clients, storing the ads, combining ads with other client responses, insufficient testing

#2 library: Some issues with running library inside the client process rather than over the network, operating independently for each client, client-specific storage, insufficient testing



Agenda

1. 6156 advertisement
2. continue design patterns: Mediator
3. Facade
4. Adapter
5. MVC
6. Strategy



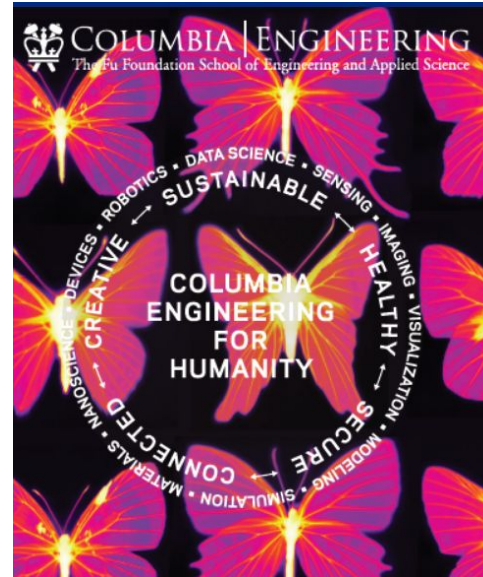
COMS E6156 Topics in Software Engineering

6156 is not "more" 4156 and is not "more advanced" 4156

6156 is a “topics” course, like 6998, where each section has a different topic

COMS E6156 Topics in Software Engineering. 3 points.

Topics in Software Engineering arranged as the need and availability arises. Topics are usually offered on a one-time basis. Since the content of this course changes, it may be repeated for credit with advisor approval. Consult the department for section assignment.



NOT Cloud Computing

Prof. Donald Ferguson teaches his very popular [Cloud Computing course](#) as a section of 6156, but my section is not about Cloud Computing



⇐ not me



⇐ not covered

What Is My Section of 6156 About?

My section is a seminar where students

- read, present and discuss research papers;
- write a midterm literature review paper; and
- conduct and write up a final research project



No lectures after first week or so, thereafter mostly student presentations and demos

Students present research papers from curated list (mostly recent best paper awards) or suggest their own (with my approval)

Students choose their own topic(s) for literature review and research project (with my approval)

Students *could* choose cloud computing as their topic, but most don't

Who Should Take 'My' 6156

Oriented towards students who want to become (or already are) researchers or technology leaders

Researchers read research papers by other researchers to *keep up to date* on their research field and to *get ideas* for their own research. They do research to try to *solve interesting problems* and prototype solutions that may, eventually, lead to real-world impact



Technology Leaders (not the same as 'tech leads') usually work in industry or government and read research papers to find ideas that could *potentially* be productized, improve products or productivity, or inform company culture, industry standards and/or government policies. They may also do small research projects to determine how *practical* it is to realize that potential for real-world impact *now*

Some Recent Student-Chosen Topics

Hallucination or Explainability? The Possibilities of Conversation-based APR

[GPT for APR = Automatic Program Repair]

DeveloperGPT [GPT for software developer command line tasks]

Adversarial Attacks on Fake News Detection Methods

NPC Population Control [NPC = Non-Player Character in video games]

A Review of Blockchain-Based Internal Tamper-Proof Relational Database Management System Designs

Overview of Methods to Treat Autism Using Software Applications

Improved Compression, Similarity Search, Clustering, Organization, and Manipulation of cDNA Libraries
Using Vector Embeddings Based on Sequence Similarity and Context [in consultation with Prof. Knowles]

Practical Security Framework [set of lab assignments for COMS W4181 Computer Security 1
in consultation with Prof. Bellare, this student TA'd 4181 the following semester where they used his labs]

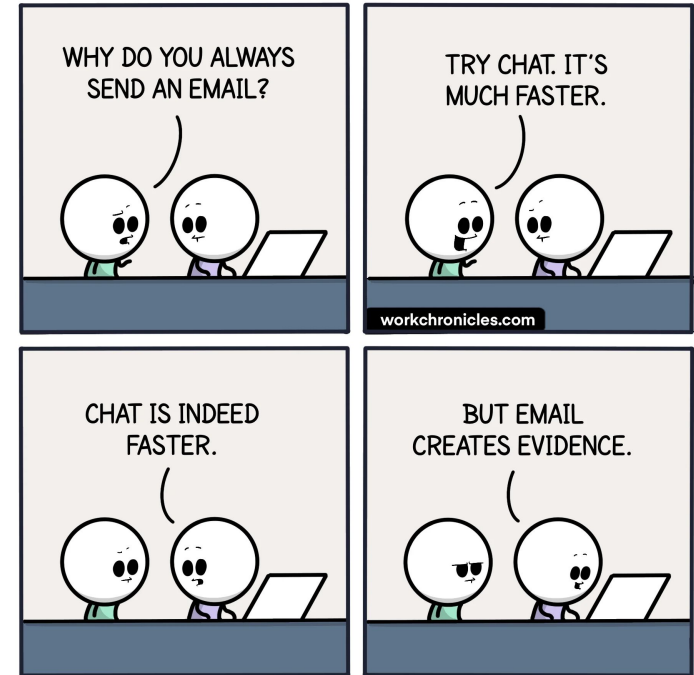


Questions?



Agenda

1. 6156 advertisement
2. continue design patterns: Mediator
3. Facade
4. Adapter
5. MVC
6. Strategy



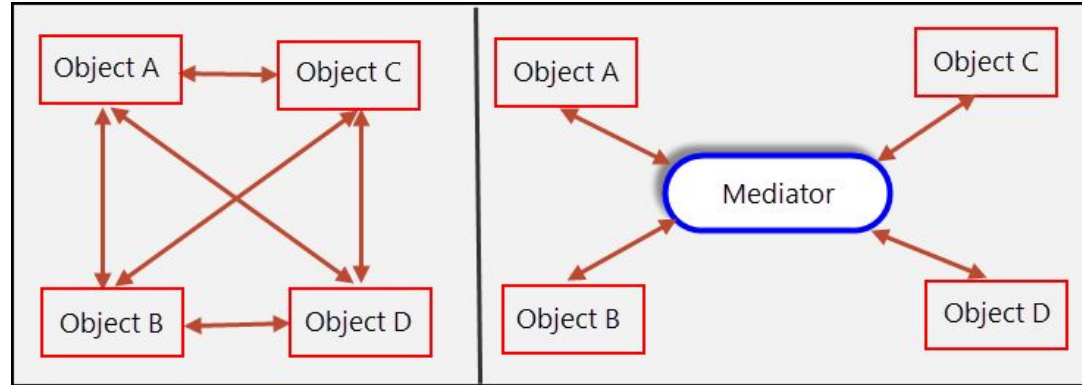
Comics about work. Made with love & lots of coffee.
Hit the [follow](#) button to get the comics in your feed.

Work Chronicles
workchronicles.com

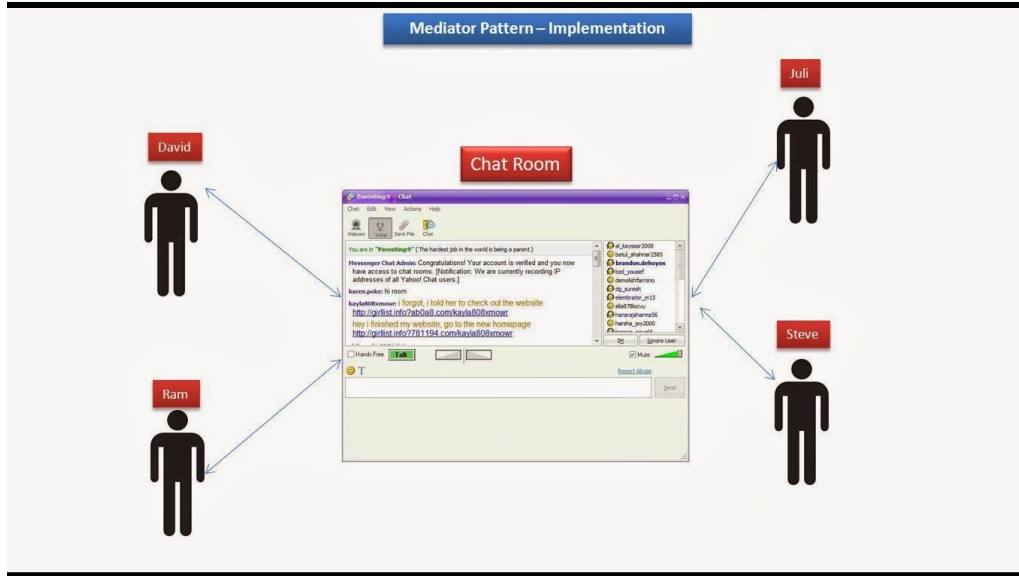
Mediator Pattern

Behavioral design pattern that

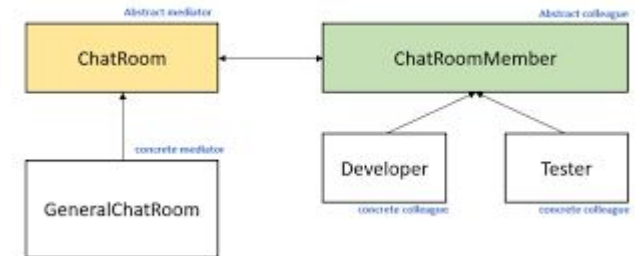
- eliminates mutual (two-way) dependencies among collection of objects,
- restricts direct communications between those objects, and
- forces them to collaborate only via a single mediator object



When to Use Mediator



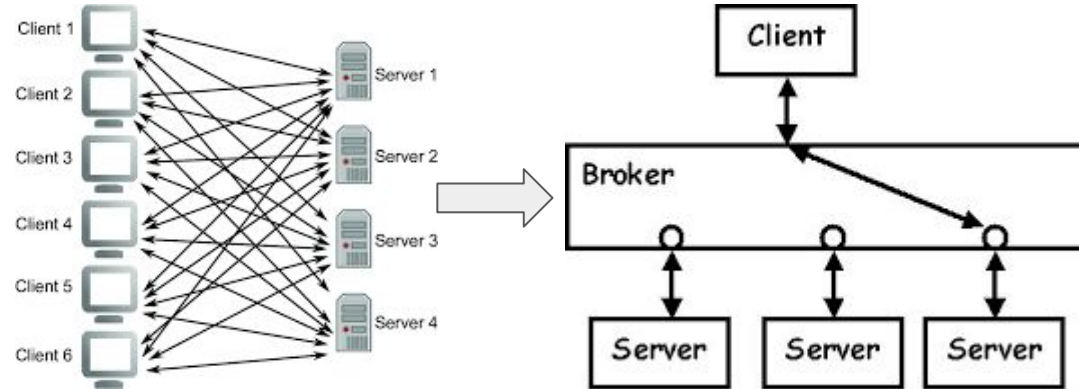
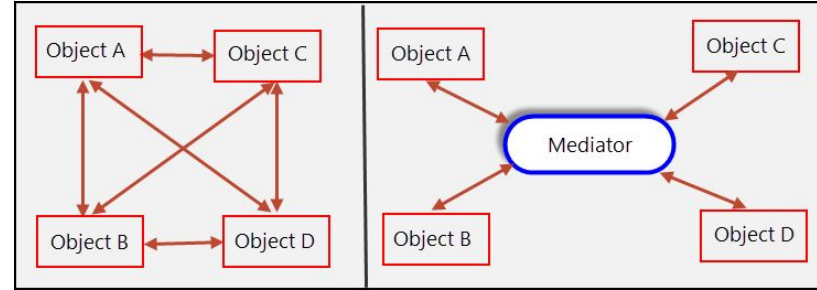
Use when some classes are tightly coupled to several other classes, so they are hard to change - the Mediator pattern lets you extract all the relationships between classes into a separate class, isolating any changes to a specific component



Mediator Pattern vs. Broker Architecture

Related to [broker architecture](#) at object instead of system granularity

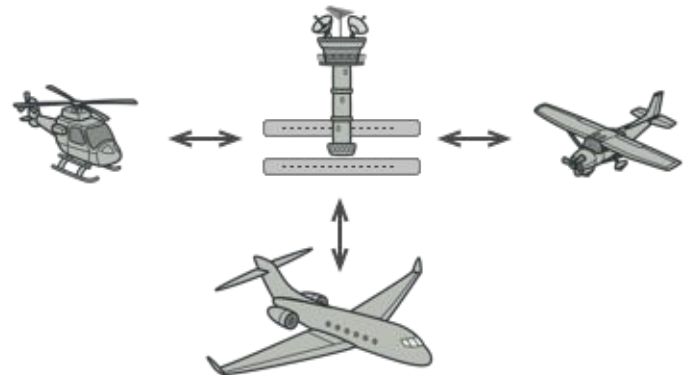
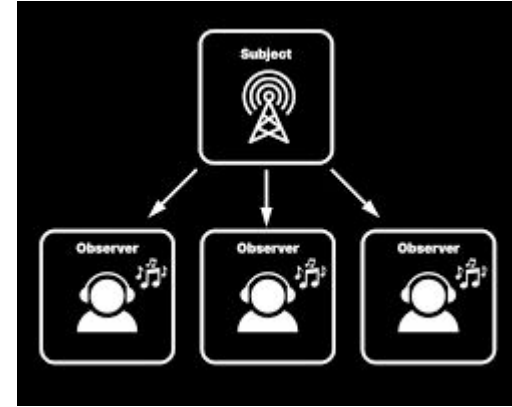
But mediator enables symmetric two-way communication among colleagues rather than request-response from clients to servers



Mediator Pattern vs. Observer Pattern

Both at object granularity

Mediator dependencies are two-way
whereas observer dependencies are one-way

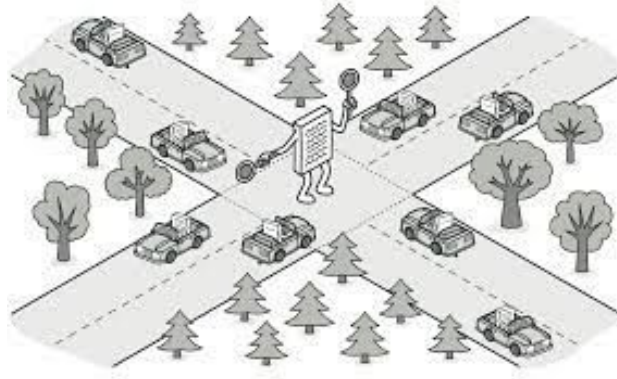


Mediator Pattern Code Examples

[C++](#)

[Java](#)

[Typescript](#)



Agenda

1. 6156 advertisement
2. continue design patterns: Mediator
3. **Facade**
4. Adapter
5. MVC
6. Strategy



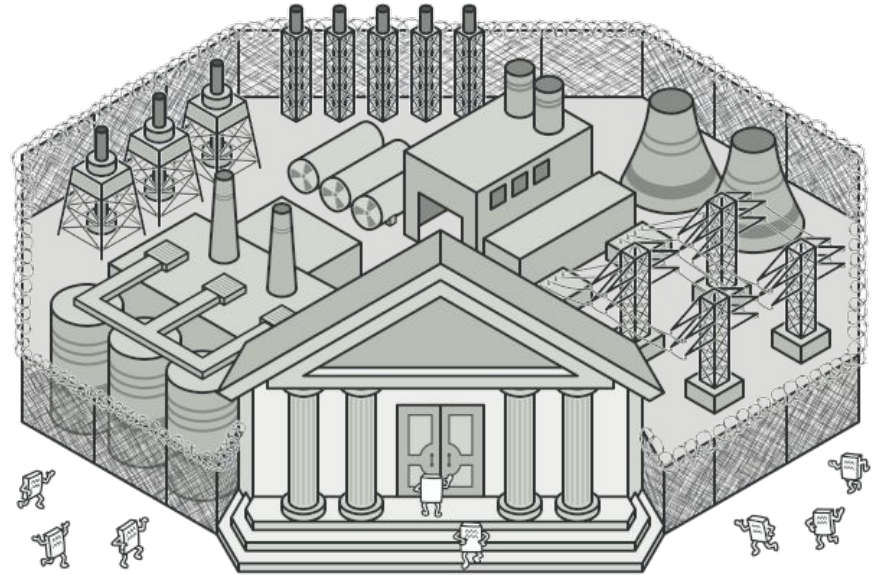
Facade Pattern is Like a Mediator for a Subsystem

Facade is a structural design pattern that provides a simplified interface to a library or other complex set of classes

Structural patterns explain how to assemble objects and classes into larger structures, while keeping the structures flexible and efficient

Facade and Mediator have similar jobs: they try to organize collaboration between lots of tightly coupled classes

Facade defines a simplified interface to a subsystem of objects, but objects *within* the subsystem can still communicate directly



Problem that Facade Addresses

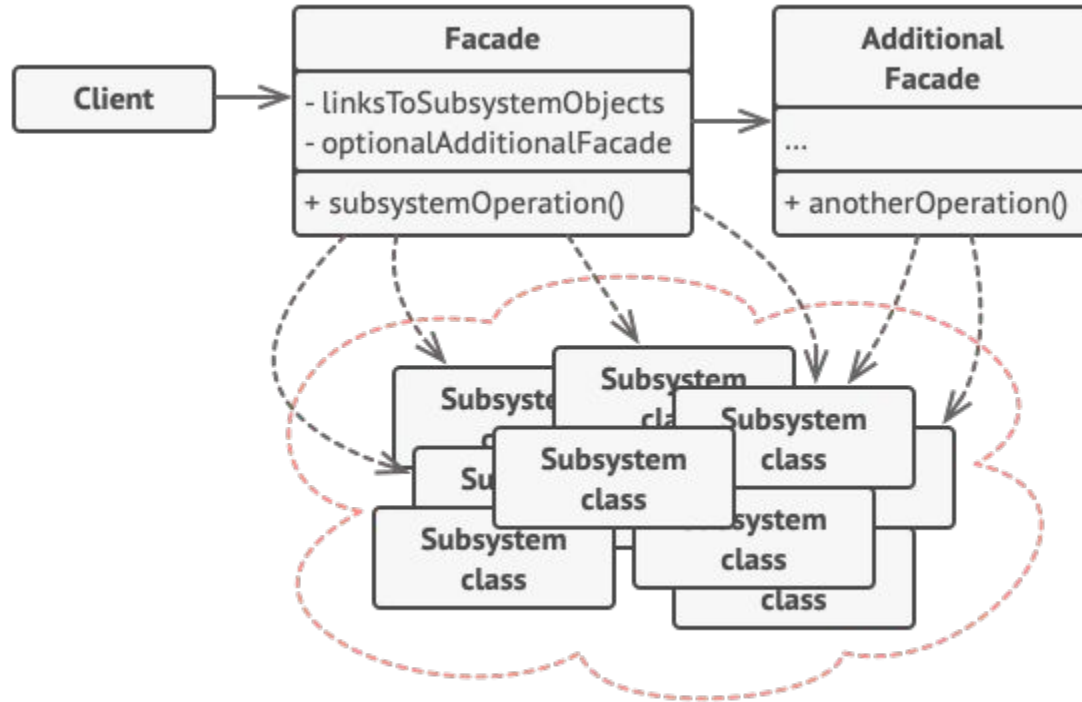


Imagine your code needs to work with a *subset* of some fancy third-party library, and you're not a power-user who needs full control of "everything"

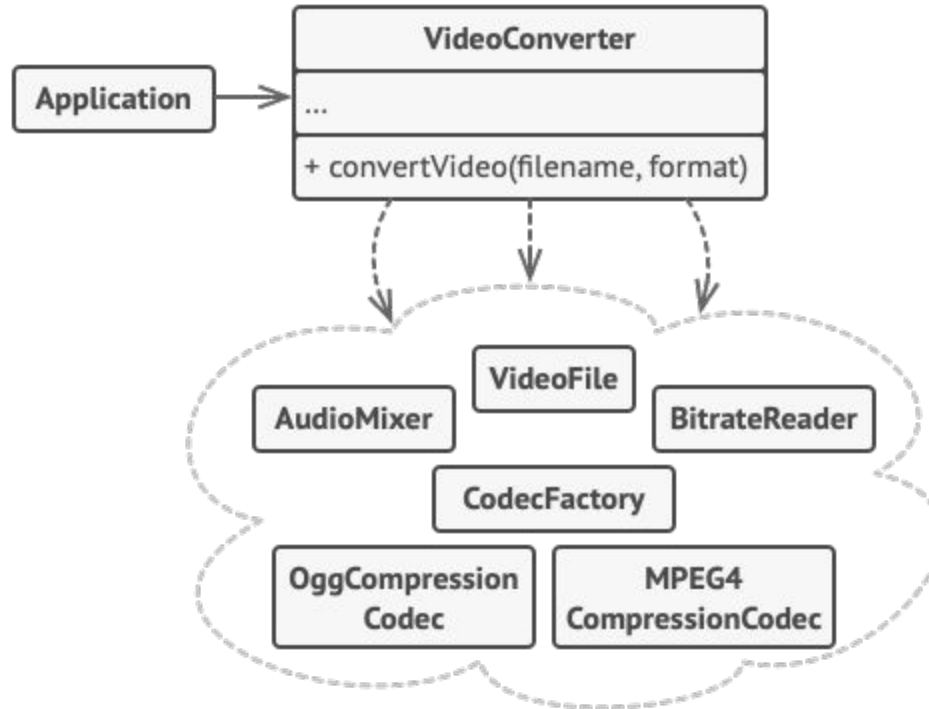
You'd still need to initialize all the third-party objects you use (and possibly others you don't), keep track of dependencies among those objects, execute method calls on different objects in the correct order, supply data in proper format that might be different for different objects, and so on

Your business logic would become tightly coupled to the implementation details of 3rd-party classes

Facade Solution



Facade Example



When to Use Facade

Use the Facade pattern when you need to have a limited but straightforward interface to a complex subsystem

Notice the Facade pattern is presented as the developer adding a facade to *someone else's* library, but alternatively the subsystem could come with multiple facades to choose from

Similarly to applying Mediator, your components become unaware of the individual components within the subsystem so can replace with a different subsystem implementing the same interface

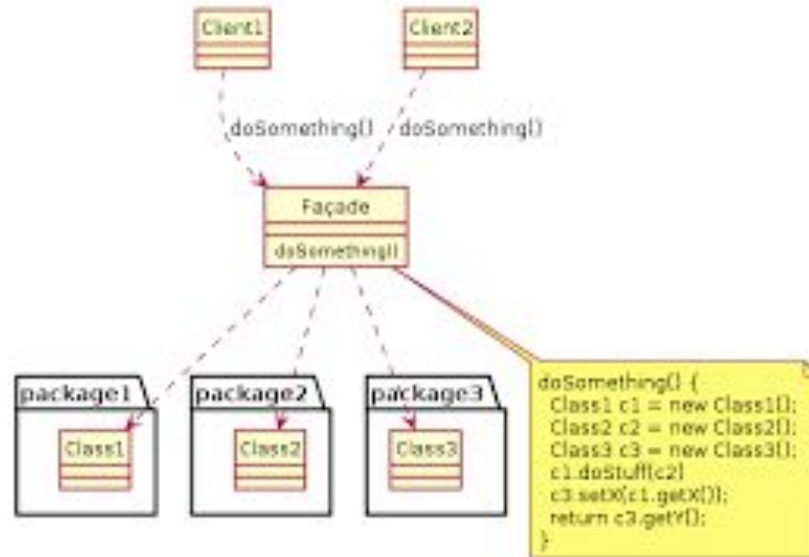


Facade Pattern Code Examples

[C++](#)

[Java](#)

[Typescript](#)



Agenda

1. 6156 advertisement
2. continue design patterns: Mediator
3. Facade
4. **Adapter**
5. MVC
6. Strategy

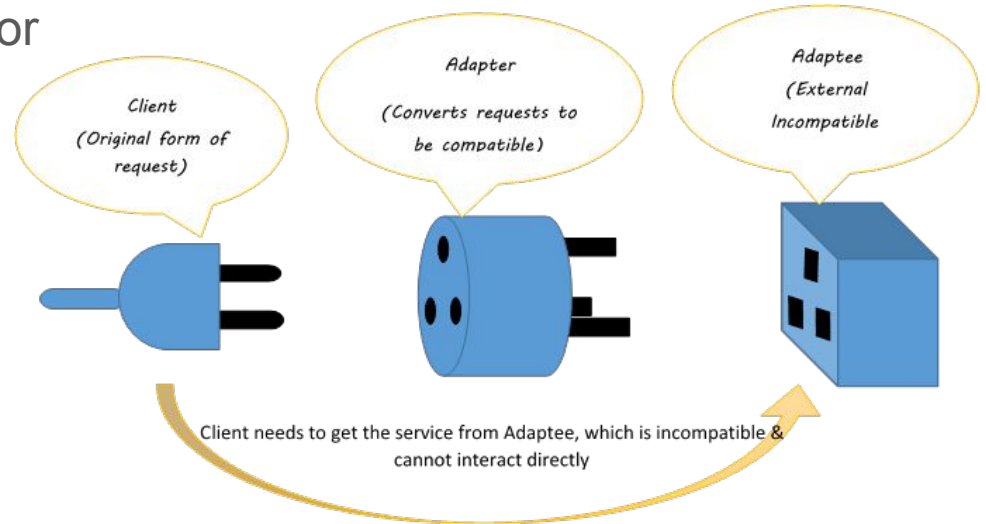
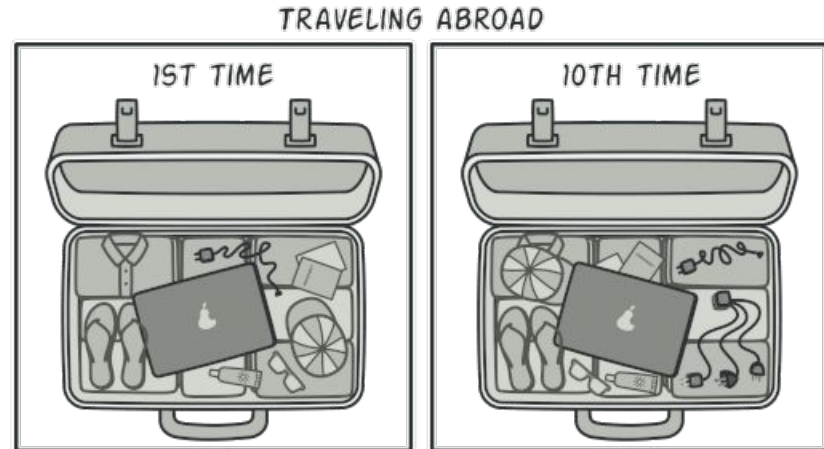


Figure 1-Adapter Pattern Concept

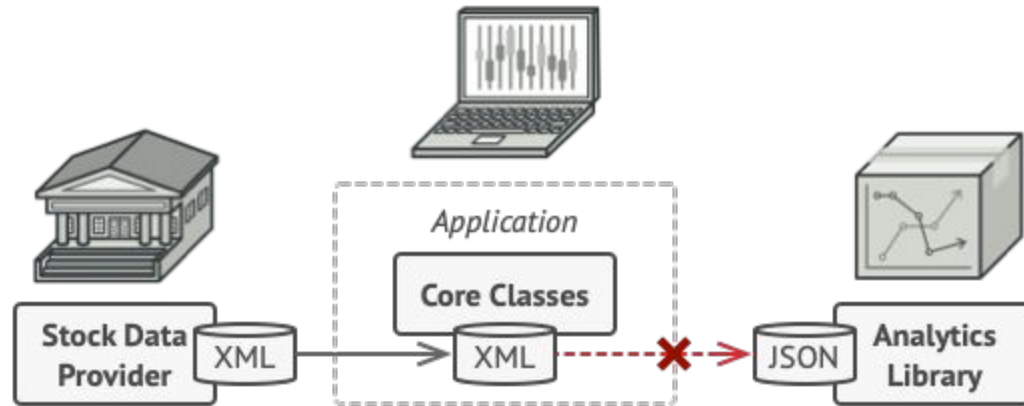
Adapter Pattern Converts Interface instead of Simplifying

Structural design pattern that allows objects with incompatible interfaces to collaborate

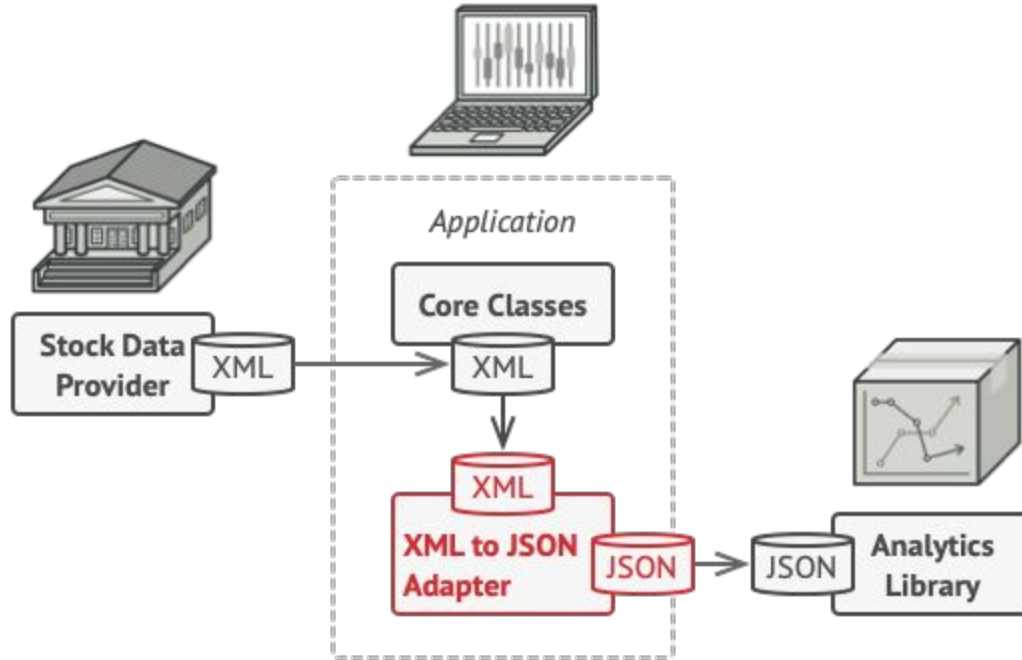
Adapter (or wrapper) usually wraps one class of objects or literally one singleton object, not a whole subsystem like Facade



Problem that Adapter Addresses



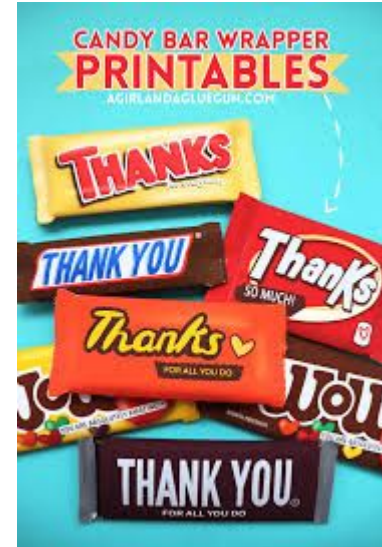
Adapter Solution



When to Use Adapter

Use when you want to use some existing class, but its interface isn't compatible with the rest of your code - the Adapter pattern creates a middle-layer class to translate between your code and a legacy class, 3rd-party class or any other class that cannot be changed

Another application is when you want to reuse several existing subclasses that lack some common functionality that can't be added to the superclass - rather than duplicate the missing functionality in the child classes, wrap inside an Adapter that implements that functionality once



Adapter Pattern Code Examples

[C++](#)

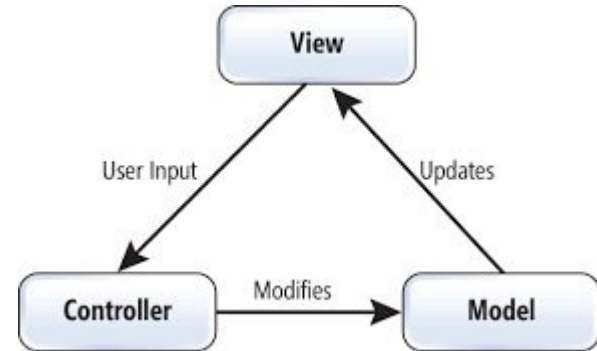
[Java](#)

[Typescript](#)



Agenda

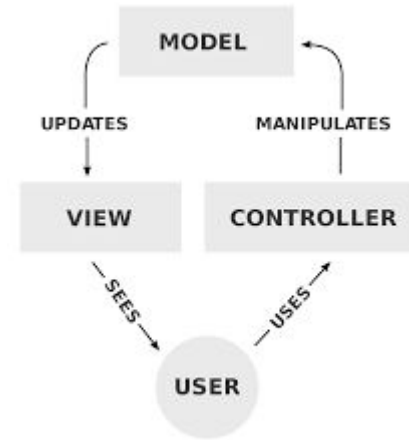
1. 6156 advertisement
2. continue design patterns: Mediator
3. Facade
4. Adapter
5. **MVC**
6. Strategy



Model-View-Controller (MVC)

MVC is usually described in terms of user-facing app, but also applies to non-UI software

- Model contains the core functionality and data,
- View displays the information to the user,
- Controller handles user input entered through the view and directs the model to 'do something'



MVC Analogy

In a restaurant, the Chef is the Model.
It's job is to take orders (requests)
from the Waiter and create food for
the Waiter to deliver to the Customer



MVC Analogy

A Waiter is a Controller. Waiters take in orders (requests), take them back to the Chef (Model), and then deliver the proper meal to Customers (Users)

The Table is the View. It just holds whatever meal the Waiter brings back from the Chef

There can be multiple controllers and views for the same model



Model

Includes business logic that manipulates state and does most of the work as well as CRUD operations for persistent state (application or domain data). Ideally a fully functional application program (with no GUI)

- Not possible to alter data except through the model
- Provides APIs to views and controllers that enable the outside world to communicate with it
- Responds to requests for information about its state (usually from the view, possibly through the controller)
- Responds to instructions to do something or change state (from the controller)



View and/or Controller may include some persistent state, such as saved layout and preferences, but the Model *owns* the domain data

View

GUI output: all CSS & HTML code goes here, updates when model changes, acts as passive observer that does not affect the model

- May be multiple views of same data, such as web, tablet and phone versions of an app
- Push - view registers callbacks with model, model notifies of changes
- Pull - view polls model for changes

Technically, MVC views do not necessarily involve a user interface, a view could be a representation 'visible' to other code



Controller

GUI input: accepts inputs from user, translates user actions on view into operations on model, decides what model should do

- May be structured as intermediary between view and model or as intermediary between user and view (renders/displays view for user to see)
- May be multiple controllers for same model

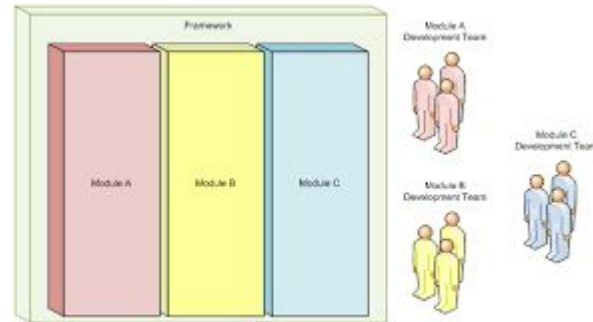
“Routes” = extension of MVC for REST APIs



“Separation of Concerns”

Important software engineering concept that you will keep hearing about

- Model is loosely coupled with the view and controller, knows nothing about either ⇒ *modularity*
- View and controller may be tightly coupled
- In OO languages usually implemented as two or three different classes that can be developed separately



MVC Example: music player app

- View - playlists, current song, graphics, sound, video, etc., with some elements that accept user selection, action, data entry
- Controller - interprets the user actions applied to view and tells model what to do
- Model - stores content, knows how to play songs, shuffle, rip, and so on

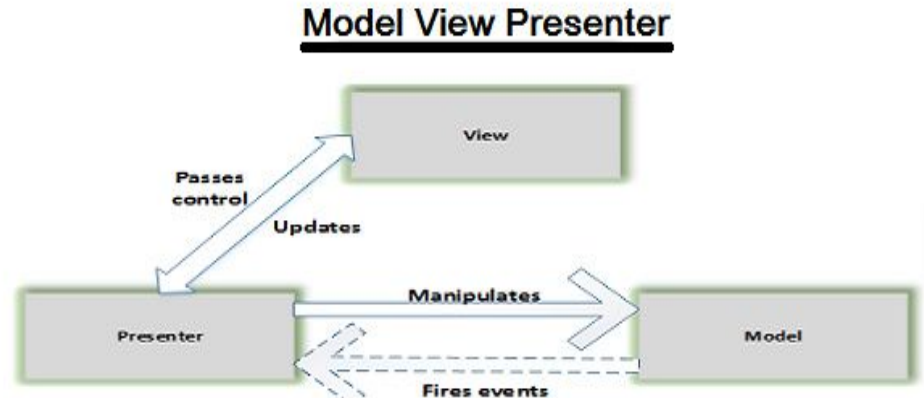
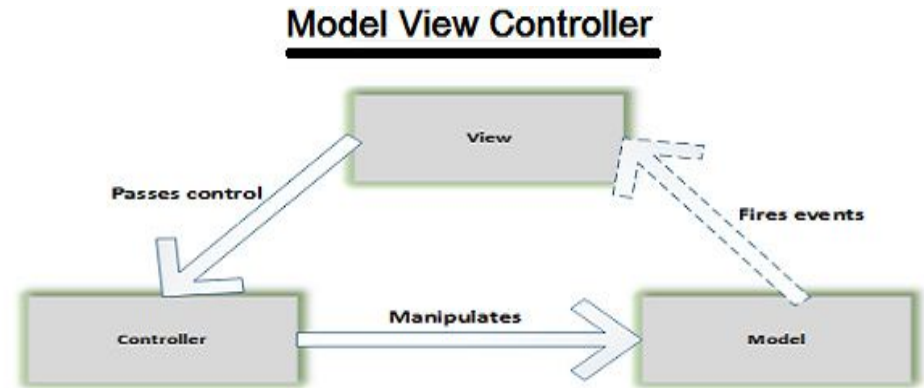


MVC Variants

Controversy over what belongs in the model

The model might be *only* the data (with possibly some bookkeeping methods) and the business logic resides in the controller (which better matches [N-tier architecture](#))

But then neither the model nor the controller is a full application by itself



How to decide?

Business logic belongs in the model when the data, and thus the model, is only part of one application

But the developer may not know in advance whether the model might eventually be used for other applications ([software sustainability](#))



Business logic belongs in the controller when the data, and thus the model, is shared across many applications

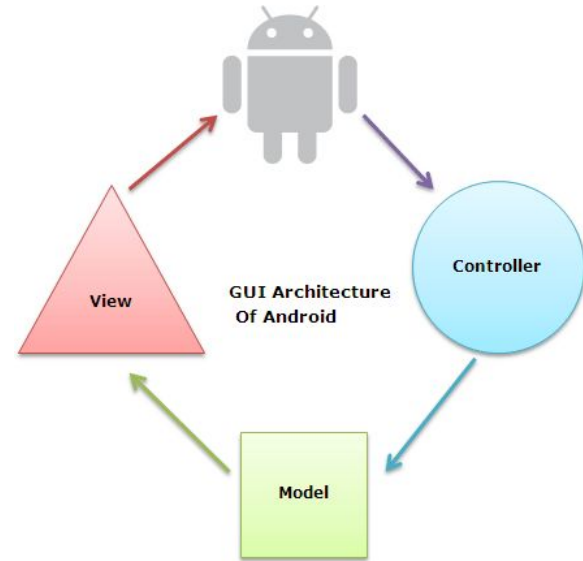
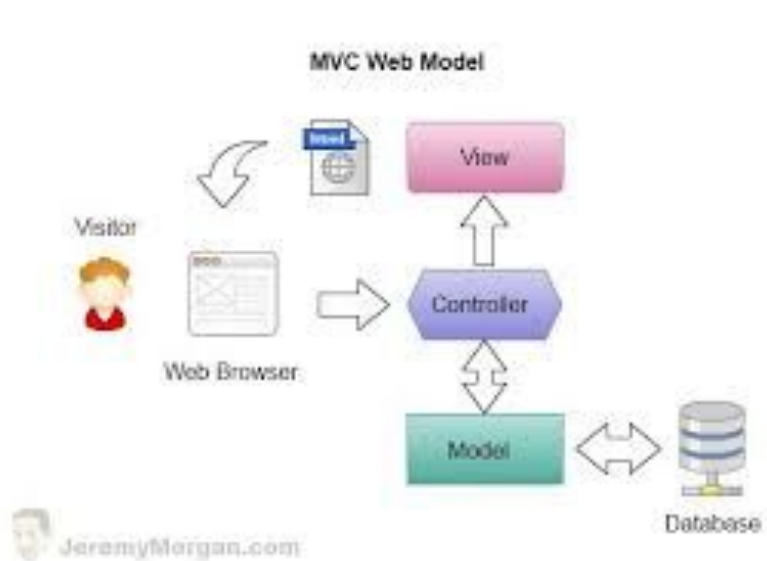
Different controller (and presumably different view or set of views) for each application

Enables replacing model while retaining controller (and views)

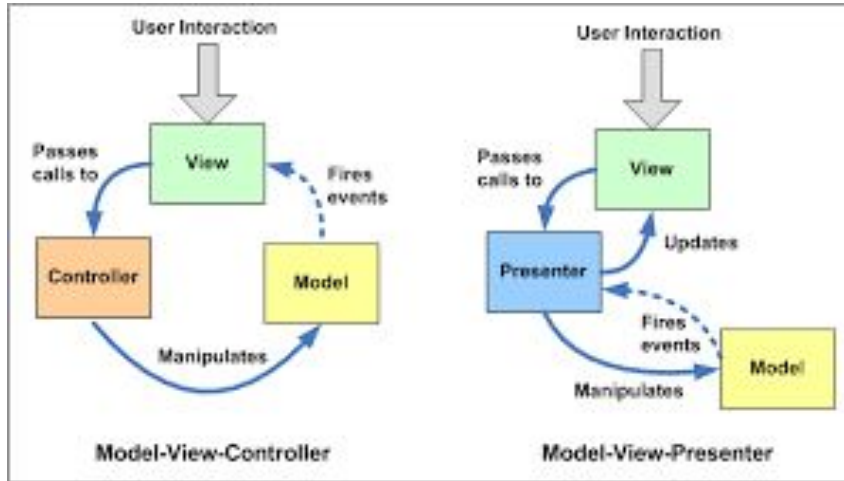
Is MVC really a design pattern?

It's not in the GoF book

But it works and everyone uses it (or something like it)



Is MVC really a design pattern?



MVC - uses Observer Pattern to passively update view

MVP - uses Observer Pattern to passively update presenter, which in turn actively updates the view

([There are other variants](#))

MVC/P is a *Compound* Design Pattern

Observer Pattern + Strategy Pattern

Observer Pattern updates the view to display the model, possibly multiple different views for the same model, e.g., desktop vs. mobile browser

Strategy Pattern supports different ways (“strategies”) to leverage the model, e.g., multiple apps using the same service



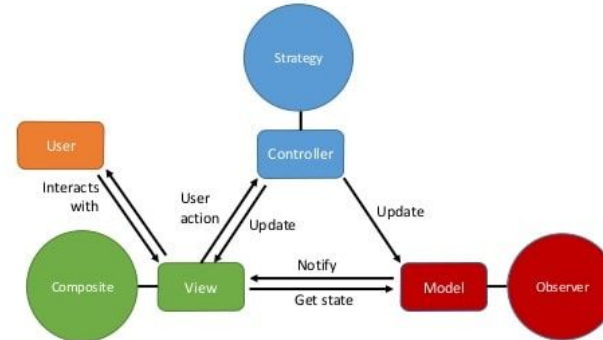
MVC Code Examples

[C++](#)

[Java](#)

[Node.js](#)

Traditional MVC & associated design patterns



1. R. Buencamino: Learning the Model-View-Controller Design Patterns in iOS. Lynda.com, 2015

Agenda

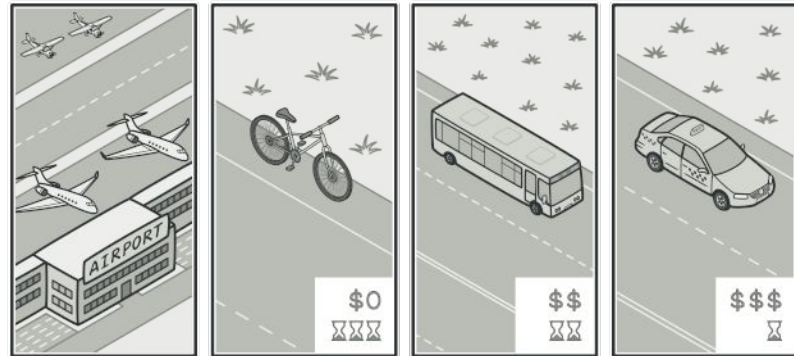
1. 6156 advertisement
2. continue design patterns: Mediator
3. Facade
4. Adapter
5. MVC
6. Strategy



What's the Strategy Pattern?

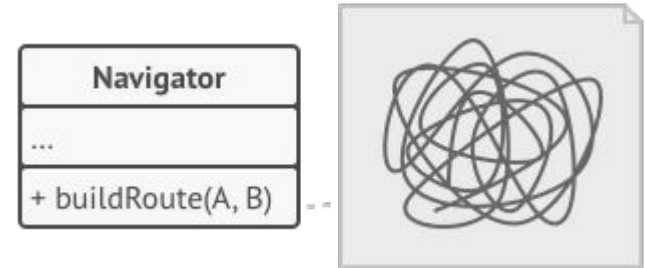
Behavioral design pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable

May allow swapping algorithms at runtime - e.g., in MVC/P an administrator might switch on the fly between privileged and regular user interfaces with different controller/presenter not just different view (e.g., [mice fake student](#))

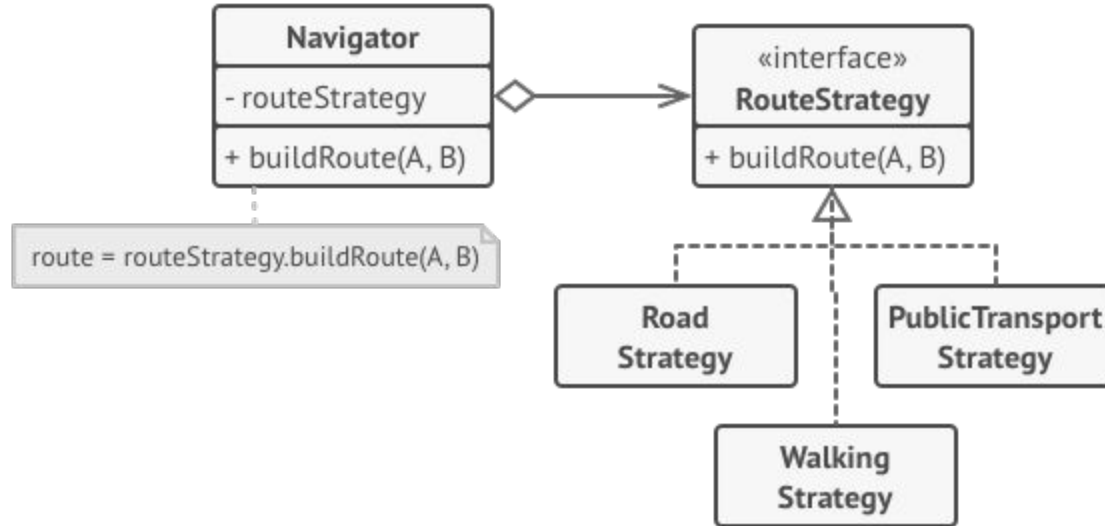


Problem that Strategy Addresses

Google maps automatic route planning



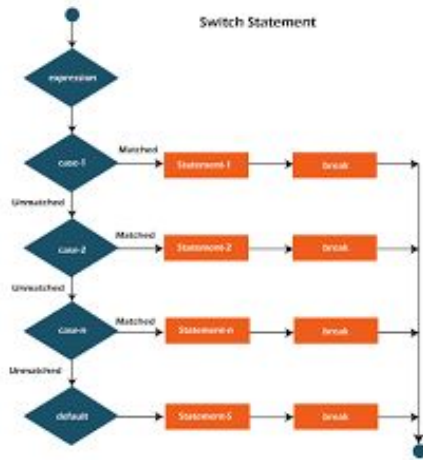
Strategy Solution



When to Use Strategy

Use when you want to use different variants of an algorithm within an object and be able to switch from one algorithm to another - the Strategy pattern lets you indirectly alter the object's behavior at runtime by associating it with different sub-objects that perform specific sub-tasks in different ways

Use when you have a lot of similar classes that only differ in the way they execute some behavior - extract the varying behavior into a separate class hierarchy and combine the original classes into one, thereby reducing duplicate code



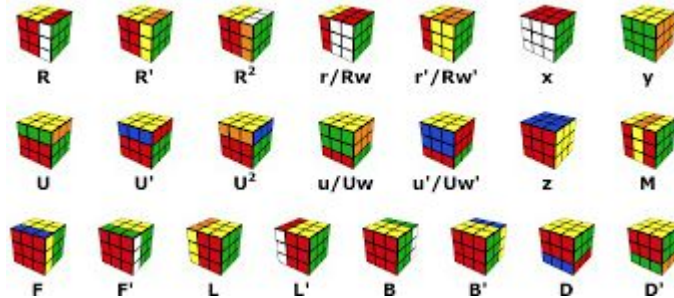
Use when your class has a massive conditional operator that switches between different variants of the same algorithm

Strategy Pattern Code Examples

C++

Java

Typescript



Many More Design Patterns

? Should we cover more of the design patterns ?

The Sacred Elements of the Faith

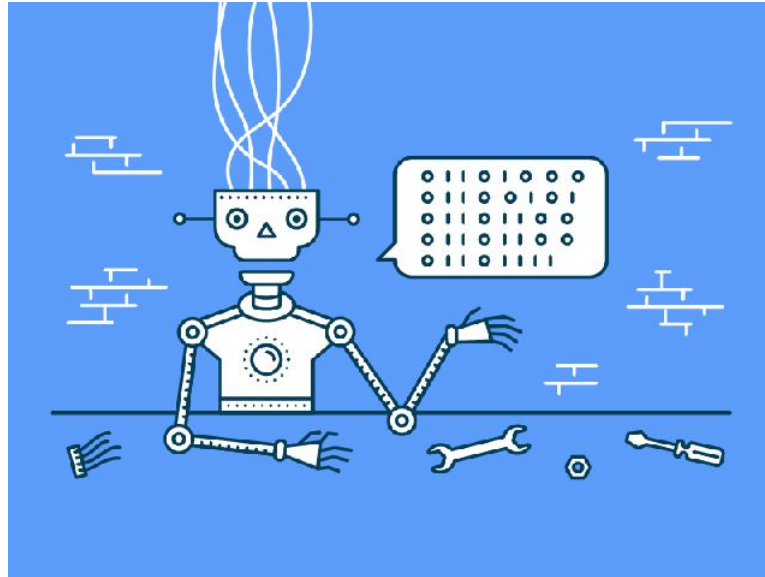
the holy
origins

the holy
structures

107 FM Factory Method	the holy behaviors					139 A Adapter	
117 PT Prototype	127 S Singleton				223 CR Chain of Responsibility	163 CP Composite	175 D Decorator
87 AF Abstract Factory	325 TM Template Method	233 CD Command	273 MD Mediator	293 O Observer	243 IN Interpreter	207 PX Proxy	185 FA Façade
97 BU Builder	315 SR Strategy	283 MM Memento	305 ST State	257 IT Iterator	331 V Visitor	195 FL Flyweight	151 BR Bridge

Upcoming Assignments

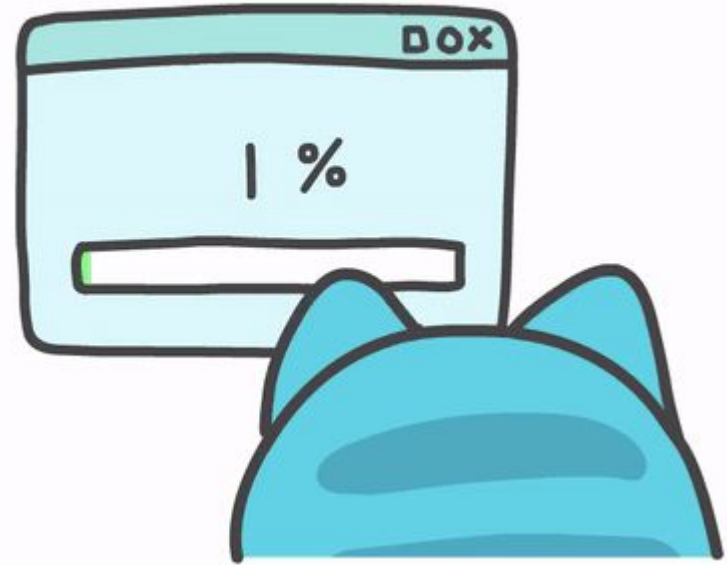
- [Second Iteration](#) due Thursday, November 30, by 11:59pm
- [Second Iteration Demo](#) due Thursday, December 7, by 11:59pm



Thursday

GCP deployment with demo

DevOps



Ask Me Anything